

PDS OS internals 13/01/2025

Domanda 1.

Si considerino le affermazioni che seguono, a proposito di possibili vantaggi e svantaggi di una inverted page table (IPT), rispetto a una tabella delle pagine standard (eventualmente gerarchica) PT. Si dica di ognuno se sia vera o falsa, motivando la risposta

Vantaggi: L'IPT permette di risparmiare di memoria:

1. Si risparmia sempre memoria:
NO. There is no saving in the case of a few processes with small address space.
2. Dipende dalle dimensioni della RAM, dal numero di processi e dal loro spazio di indirizzamento virtuale
YES, for similar reasons to answer 1
3. Si risparmia sempre memoria quando lo spazio di indirizzamento di un processo è maggiore della dimensione della RAM
NO, though this is a case where IPT can gain over PT, there are other cases as well: e.g. many processes with address space smaller than RAM, but the union of all Page Tables bigger than the IPT.
4. Si può risparmiare anche in casi in cui lo spazio di indirizzamento di ogni processo è inferiore alla dimensione della RAM
YES. Because the IPT is unique for all processes.

Svantaggi: L'IPT è lenta, perché non garantisce accesso diretto ma occorre una ricerca

1. La chiave di ricerca è la coppia (pid,frame)
NO. The frame corresponds to the index No
2. La chiave di ricerca è la coppia (pid,pagina)
YES, if it is implicitly understood that the search key "includes" the page index. NO, if you consider that it is not enough (there is also the pid)
3. Per migliorare le prestazioni si sostituisce la IPT con una tabella di HASH
NO. It would just be a replacement. Or rather, if you replace it with a HASH table it is another solution (not an IPT any more)
4. Per migliorare le prestazioni si aggiunge alla IPT una tabella di HASH
YES. This is exactly what is usually done to improve the performance of IPT

Sia dato un processo avente spazio di indirizzamento virtuale di 48 GB, in un sistema dotato di 16GB di RAM, con architettura a 64 bit (in cui si indirizza il Byte) e gestione della memoria paginata (pagine/frame da 4KB). Si supponga che 4GB di RAM sia allocato in modo statico al kernel. Si vogliono confrontare una soluzione basata su tabella delle pagine standard (una tabella per ogni processo) e una basata su IPT. Si calcolino

- A) le dimensioni della tabella delle pagine (a un solo livello) per il processo e della IPT. Si ipotizzi che il pid di un processo possa essere rappresentato su 12 bit. Si utilizzino 28 bit per gli indici di pagina e/o di frame (nella PT o nella IPT) e si tenga conto che, per allineamento, una cella di IPT o PT può solo essere di 32 o 64 bit.

Size of PT and IPT

Number of pages = $48\text{GB}/4\text{KB} = 12\text{M}$ (corresponds to the number of cells/entries in the PT)

Lower bound for x:

$$2^x \geq 48\text{GB}/4\text{KB}$$

$$x \geq \text{ceil}(\ln(12\text{M})) = 24, \text{ so the lower bound is 24 bits.}$$

Page Table standard:

One PT entry has size 4B (it has to include 24 bits + 4 extra bits, so due to alignment we choose 32 bits)

$$|PT| = 12M * 4B = \mathbf{48MB}$$

IPT

The table, unique for all processes, has fixed size, because it has one entry for each RAM frame. So it does not depend on a process's size. It is enough to represent the "user" RAM the in the IPT: $12GB = 16GB - 4GB$ (we exclude the RAM allocated to the kernel).

Every IPT row needs at least 12bits (pid) + 24 bits (page) + 4 (extra) = 40 bits. So due to alignment we go to 64bits = 8B.

$$N \text{ frame} = 12GB / 4KB = 3M$$

$$|IPT| = 3M * 8B = \mathbf{24MB}$$

- B) Si dica infine, utilizzando la IPT proposta (12 bit di pid, 28 bit per un indice di pagina/frame), quale è la dimensione massima possibile per lo spazio di indirizzamento virtuale di un processo.

Address space size

NOTE

In this case, the 64-bit address cannot be fully used, as there is a limit of 24 bits for the page indexes.

The maximum number of virtual pages of a process is therefore limited by the size of the page indexes (24 bits): this number is therefore $2^{24} = 16M$.

Since each page is 4KB in size, the virtual address space has a maximum size

$$|\text{address space}| = (16M) * 4KB = 64GB.$$

Domanda 2.

Sia dato un disco organizzato con blocchi fisici e logici di dimensione 8KB. Il disco contiene più partizioni: la partizione A, di NB blocchi, è formattata per un file system che alloca staticamente NM blocchi per i metadati (che includono directory, file control blocks e una bitmap per la gestione dello spazio libero) e ND blocchi per i dati dei file. La bitmap ha un bit per ciascuno degli ND blocchi di dati. NM/4 blocchi di metadati sono riservati alla bitmap.

Si risponda alle seguenti domande:

- A) Si calcoli il rapporto ND/NM.

$$|\text{bitmap}| = NM/4 \text{ blocks} = NM/4 * 8K * 8 \text{ bits} = 16K * NM \text{ bits}$$

each bitmap bit corresponds to one of the ND data blocks

$$ND = 16K * NM$$

$$ND/NM = 16K$$

- B) Supponendo che la bitmap indichi un rapporto blocchi liberi / usati del 33,33% (quindi 1 blocco libero ogni 3 usati), si calcoli (in funzione di NM) la dimensione massima per un intervallo contiguo di blocchi liberi, assumendo la configurazione più favorevole della bitmap (favorevole significa in grado di ottenere partizioni libere più grandi). Si dia la stessa risposta anche assumendo la configurazione della bitmap meno favorevole.

$$N_{\text{free}}/N_{\text{used}} = 1/3 \Rightarrow N_{\text{free}} = 0.25 \cdot (N_{\text{free}} + N_{\text{used}}) = 0.25 \cdot (N_{\text{bits}}) = 0.25 \cdot 16K \cdot NM \text{ bits} = 4K \cdot NM \text{ bits}$$

The most favourable situation is when all free blocks are contiguous:

$$N_{\text{good}} = 4K \cdot NM$$

The less favourable situation is when all free blocks have no other adjacent free block:

$$N_{\text{bad}} = 1$$

- C) Si supponga che un file control block (FCB) abbia dimensione 256B e NM/4 blocchi di metadati siano riservati agli FCB, per un massimo di 16K file. Si calcolino ND, NM e NB. Si esprima anche la dimensione della bitmap e della partizione A, espressa in Byte.

The maximum number of files is 16K

A block can contain $8KB/256B = 32$ FCBs

The maximum number of FCBs is $32 \cdot NM/4 = 8 \cdot NM$

As files correspond to FCBs:

$$8 \cdot NM = 16K$$

$$NM = 2K$$

$$ND = 32K \cdot NM = 64M$$

$$NB = ND + NM = 64M + 2K$$

$$|A| = (64M + 2K) \cdot 8KB = 512GB + 16MB$$

$$|bitmap| = NM/2 \cdot 8KB = 8MB$$

Domanda 3.

Si risponda alle seguenti domande sulla gestione della memoria:

- A) Si consideri il caricamento dinamico (dynamic loading) e il link dinamico (dynamic linking). È possibile caricare dinamicamente un programma senza che sia necessario il dynamic linking? Il dynamic linking richiede che un programma sia anche caricabile dinamicamente (dynamic loading)?

Yes. Although dynamic linking can be combined with dynamic loading, it is not mandatory: a program can be statically linked and dynamically / incrementally loaded, eg. program-based dynamic loading.

Note: dynamic load means that pieces of a program are loaded into memory only if / when needed, dynamic link means that references between modules are resolved at runtime (especially useful for shared libraries).

- B) Si spieghi brevemente perché un'Inverted Page Table necessiti di una tabella di HASH e si spieghi perché la soluzione di IPT + tabella di HASH è diversa da una soluzione con PT basata solamente su tabella di Hash?

Because if you didn't use a HASH table you would need a linear search of the logical page number (p) in the IPT.

The two solutions differ because, despite being very similar in terms of performance of the HASH table, in one case (with the IPT, the IPT entries are inserted directly into the chaining lists of the HASH table, while in the case of simple HASH the classic allocations and deallocations of elements are required at each insertion / cancellation from the HASH. So IPT+HASH solution is more efficient in the use of memory.

- C) Si consideri una CPU dotata di TLB: la TLB può contenere entry di più processi simultaneamente o è vincolata a contenere entry per un solo processo? Il "valid" bit presente in un entry della TLB è una semplice copia del "valid" bit presente nella Page Table (motivare)?

Both types of TLB exist: the ones containing entries for multiple processes and the one containing just entries for the currently active process. The latter ones need a TLB cleanup/reset at each context switch. The valid bit has different meanings in the TLB and in the PT: in the TLB it means that the TLB entry is unused/free, whereas in the PT it means that the logical page is not mapped to a frame.

Domanda 4.

È dato un Sistema OS161. Si supponga di aver aggiunto le istruzioni seguenti a `kern/conf/conf.kern`

```
defoption project
optfile project syscall/project.c
```

e di aver creato il file `PROJECT` in `kern/conf`, copiato dal file `DUMBVM`.

- A) Si dica se le azioni descritte nel seguito, più l'esecuzione (in `kern/conf`) di `./config PROJECT`, sono sufficienti affinché il file opzionale `syscall/project.c` sia compilato quando si eseguono (in `kern/compile/PROJECT`) i comandi

```
bmake depend
bmake
```

NO. They are not enough. The instructions define the option and the dependency of the file `syscall/project.c` on the option. But you need to enable this option, in the `PROJECT` file. Otherwise the `project.c` file will be not only not compiled, but not even considered in the generation of dependencies.

- B) Quale file, tra `project.h` e `opt-project.h` viene generato automaticamente dal comando `./config PROJECT`? Il file viene sempre generato, oppure solo se l'istruzione seguente compare nel file `PROJECT`?

```
options project
```

`opt-project.h`. The file is generated even if `PROJECT` does not contain `options project`. In order for the `.h` file to be generated, it is sufficient for `conf.kern` to contain the definition of the `project` option. A possible `project.h` file can, if necessary/appropriate, be created "manually", and must be added, as optional or not, as appropriate, in `conf.kern`.

C) Cosa contiene il file (quello generato automaticamente, citato nella domanda precedente)?

```
opt-project.h, apart from multiple inclusion protection, contains only one directive to the pre-compiler:  
  
#define OPT_PROJECT 1  
  
or  
  
#define OPT_PROJECT 0  
  
depending on whether or not PROJECT contains the options project statement
```

D) Si supponga di inserire in `main.c` l'istruzione

```
project_init();
```

Considerando che la funzione `project_init()` e' implementata nel file `syscall/project.c`, come si puo' fare in modo che l'istruzione sia considerata e compilata solo nelle versioni del kernel in cui l'opzione `project` e' abilitata?

```
Conditional compilation must be used, using the OPT_PROJECT macro, defined in opt-project.h. So the header of the main.c file (or one of the .h included by it) must contain
```

```
#include "opt-project.h"
```

```
and the instruction to be conditioned (in main.c) will be written as
```

```
#if OPT_PROJECT  
project_init();  
#endif
```

Domanda 5.

Si consideri un processo user OS161 (a ogni domanda si dia una risposta e una motivazione).

A) Quale, tra `trapframe` e `switchframe`, viene usato per gestire una `system call`?

```
A trapframe is used, because a system call is handled by means of a trap/interrupt protocol (a system call is a sort of software interrupt)
```

B) Un `trapframe` viene allocato nello `user stack` o nello `stack` a livello `kernel` di un processo?

```
In the kernel stack, as the trapframe is a kernel data structure (although used to handle the user process). It would be a problem leave it visible in user mode, as a user program could corrupt it.
```

D) Perché i dispositivi di IO sono mappati in `kseg1`? È possibile che un dispositivo di IO sia mappato in `kseg0`?

IO devices need to be mapped in the kernel space. They are mapped to kseg1 as it is not cached: an IO device cannot be read/written using the cache, as any read/write operation needs to be performed (directly) on the IO device. For the same reason, the device cannot be mapped to kseg0, which is cached.

D) Cosa succede se un processo user chiama `read(fd, buf, nb)`, dove `buf` é un indirizzo in `kseg0`? Cosa dovrebbe fare la system call `SYS_read`, al fine di gestire correttamente questo caso?

This is clearly a programming error, as the destination of a read (un a user program) iis a kernel address. The the `SYS_read` syscall should properly check and handle the error, and avoid performing the reads operation.